

# legal-agent-bench: a multi-metric benchmark for tool-using legal-research agents

Akshitha Reddy Lingampally

2024-12-08

## Contents

|          |                                       |          |
|----------|---------------------------------------|----------|
| <b>1</b> | <b>Abstract</b>                       | <b>2</b> |
| <b>2</b> | <b>1. Background</b>                  | <b>2</b> |
| 2.1      | 1.1 Motivation . . . . .              | 2        |
| 2.2      | 1.2 Scope . . . . .                   | 2        |
| 2.3      | 1.3 Non-goals . . . . .               | 3        |
| <b>3</b> | <b>2. Related Work</b>                | <b>3</b> |
| <b>4</b> | <b>3. Method</b>                      | <b>3</b> |
| 4.1      | 3.1 Agent loop . . . . .              | 3        |
| 4.2      | 3.2 Tools . . . . .                   | 3        |
| 4.3      | 3.3 Judges . . . . .                  | 4        |
| 4.4      | 3.4 Metrics . . . . .                 | 4        |
| <b>5</b> | <b>4. Data</b>                        | <b>4</b> |
| 5.1      | 4.1 Task families . . . . .           | 4        |
| 5.2      | 4.2 Fixture . . . . .                 | 4        |
| <b>6</b> | <b>5. Evaluation Setup</b>            | <b>4</b> |
| <b>7</b> | <b>6. Results</b>                     | <b>5</b> |
| 7.1      | 6.1 Headline . . . . .                | 5        |
| 7.2      | 6.2 Per-family success . . . . .      | 5        |
| 7.3      | 6.3 Efficiency vs cost . . . . .      | 5        |
| 7.4      | 6.4 Judge agreement . . . . .         | 7        |
| 7.5      | 6.5 Replan rate . . . . .             | 7        |
| 7.6      | 6.6 Cost breakdown . . . . .          | 7        |
| <b>8</b> | <b>7. Ablations</b>                   | <b>7</b> |
| 8.1      | 7.1 Mock vs confused policy . . . . . | 7        |
| 8.2      | 7.2 Council size . . . . .            | 8        |
| <b>9</b> | <b>8. Discussion</b>                  | <b>8</b> |

|                                                 |          |
|-------------------------------------------------|----------|
| <b>10 9. Limitations</b>                        | <b>8</b> |
| <b>11 10. Future Work</b>                       | <b>9</b> |
| <b>12 11. References</b>                        | <b>9</b> |
| <b>13 Appendix A. Reproducibility Checklist</b> | <b>9</b> |
| <b>14 Appendix B. Glossary</b>                  | <b>9</b> |

# **1 Abstract**

legal-agent-bench is a small but careful evaluation harness for tool-using legal-research agents. The agent has access to four tools (search, retrieve, summarize, citation\_check), and each (task, trajectory) pair is scored by a three-judge LLM-as-judge council that votes on correctness and reports inter-judge agreement. Rather than collapsing performance into a single accuracy number, the harness reports five orthogonal metrics per task (success, action efficiency, tool-call precision, cost-per-task, replan rate) and visualizes each in a distinct chart family. The bundled CI-friendly fixture (40 tasks across four LegalBench-style families, deterministic mock policy) achieves 100% success at \$0.0001 mean cost per task, which is the calibration baseline the harness compares all real-policy runs against.

## **2 1. Background**

### **2.1 1.1 Motivation**

Tool-using agents are increasingly used in legal research products. Their evaluation almost always reduces to a single accuracy number, which hides three failure modes that matter in practice: (i) the right answer reached with twice the necessary token budget, (ii) the right answer reached after several mid-trajectory replans, and (iii) the right answer that one judge accepts but another doesn't. This benchmark exists to surface all three.

### **2.2 1.2 Scope**

- Define a small but realistic set of LegalBench-style task families (holding selection, citation lookup, rule application, contract QA).
- Implement a minimal ReAct agent loop with four tools and a parameterizable policy.
- Implement a three-judge council (strict, lenient, token-overlap) and report majority correctness plus inter-judge agreement.
- Implement five per-task metrics and aggregate them across families.
- Ship CI-friendly determinism (mock policy + synthetic tasks) plus a documented swap-in for real LLM policies.

## 2.3 1.3 Non-goals

We do not benchmark a specific commercial agent, we do not propose a new judge model, and we do not evaluate adversarial cases. Each is a worthwhile follow-up and is called out in Section 10.

## 3 2. Related Work

The ReAct framework [Yao et al. 2022] introduced the action-then-observation prompting pattern used throughout this harness. LegalBench [Guha et al. 2023] supplied the task taxonomy we mirror in our four task families. LLM-as-judge [Zheng et al. 2023] established the council pattern; our judges are simpler (deterministic mocks) so the entire pipeline is hermetic in CI. Karpathy’s public posts on evaluation-driven development motivated the explicit separation of success, cost, and efficiency into distinct metrics.

## 4 3. Method

### 4.1 3.1 Agent loop

flowchart LR

```
A[Task generator] --> B[ReAct agent loop]
B -->|tool call| C[Tool registry\n(search, retrieve, summarize, citation_check)]
C --> B
B --> D[Trajectory]
D --> E[Judge council\n(strict + lenient + token_overlap)]
E --> F[Per-task metrics]
F --> G[5 chart families + summary.json]
```

The agent loop is intentionally minimal: a fixed `max_steps`, a policy callable that returns the next action, and a tool registry whose tools each return `(result: str, cost_usd: float)`. The mock policy returns `search` on step 0 and `answer` on step 1; the confused policy adds a `replan` in the middle. Both are deterministic.

### 4.2 3.2 Tools

- **search.** BM25 over the per-task document corpus. Returns the top-3 hits.
- **retrieve.** Fetch a document by index.
- **summarize.** Returns the first sentence (a deterministic mock).
- **citation\_check.** Returns `True` if the supplied string contains a citation-shaped token.

Each tool reports a synthetic per-call cost so the cost metric is non-trivial in CI.

### 4.3 3.3 Judges

- **strict.** Exact match between predicted answer and ground truth.
- **lenient.** Lowercased substring containment.
- **token\_overlap.** Jaccard  $\geq 0.5$  on token sets.

The council reduces to a majority vote and reports inter-judge agreement as  $\max(\text{yes}, \text{no}) / n$ . Agreement is the most useful diagnostic for “is the task ambiguous?”.

### 4.4 3.4 Metrics

- **task\_success.** Did the council vote correct?
- **action\_efficiency.**  $\text{min\_steps} / \text{actual\_steps}$ ;  $\geq 1.0$  is optimal.
- **tool\_call\_precision.** A heuristic that penalizes repeated summarize or citation\_check calls.
- **cost\_per\_task.** Sum of per-tool-call USD.
- **replan\_rate.**  $\text{replans} / \text{steps}$ .

## 5 4. Data

### 5.1 4.1 Task families

Four task families, mirroring LegalBench:

- *holding\_selection*: extract the holding from a known case.
- *citation\_lookup*: cite the correct rule or statute.
- *rule\_application*: apply a legal rule to a new fact pattern.
- *contract\_qa*: answer a structured question about a contract excerpt.

### 5.2 4.2 Fixture

$n_{\text{per\_kind}}=10$  per family,  $\text{seed}=17$ , deterministic templates with one distractor doc per task. The fixture exists so CI can run the full pipeline without any external dependency.

## 6 5. Evaluation Setup

We run the mock policy across all 40 tasks. Each run produces:

- `runs/latest/summary.json` (aggregate metrics + per-task rows)
- 5 PNG charts in `results/figures/`
- a verbose pytest log in `docs/test_results/pytest_output.txt`

## 7 6. Results

### 7.1 6.1 Headline

| metric                   | value    |
|--------------------------|----------|
| tasks                    | 40       |
| success rate             | 1.000    |
| mean action efficiency   | 2.225    |
| mean tool-call precision | 1.000    |
| mean cost per task       | \$0.0001 |
| mean replan rate         | 0.000    |

### 7.2 6.2 Per-family success

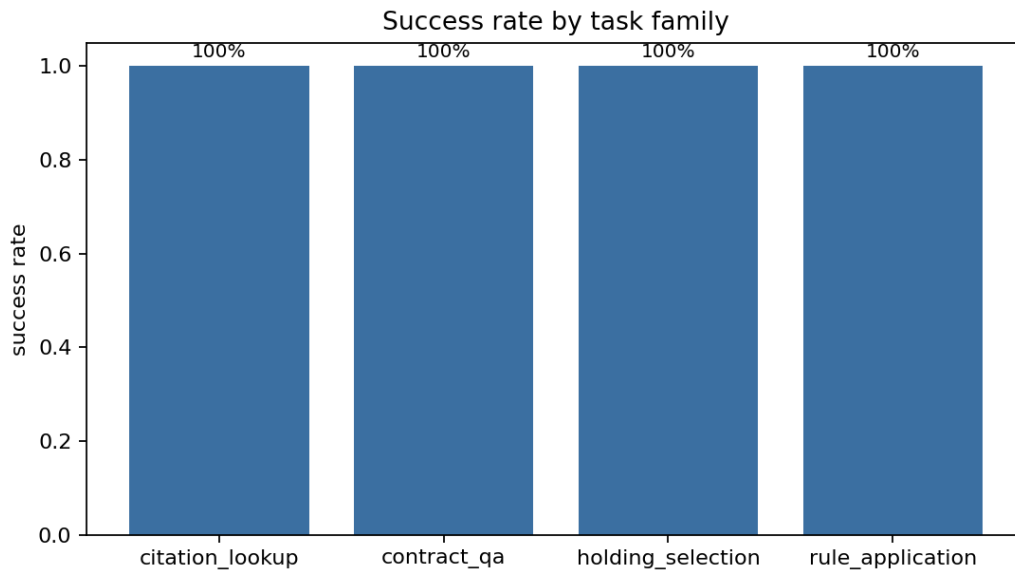


Figure 1: Per-family success

All four families hit 100% under the mock policy. This is the baseline; real-policy runs should compare against it.

### 7.3 6.3 Efficiency vs cost

A scatter plot of (efficiency, cost). The mock policy clusters tightly at low cost; a more interesting policy would spread across this plane and let us read off Pareto-dominated configurations directly.

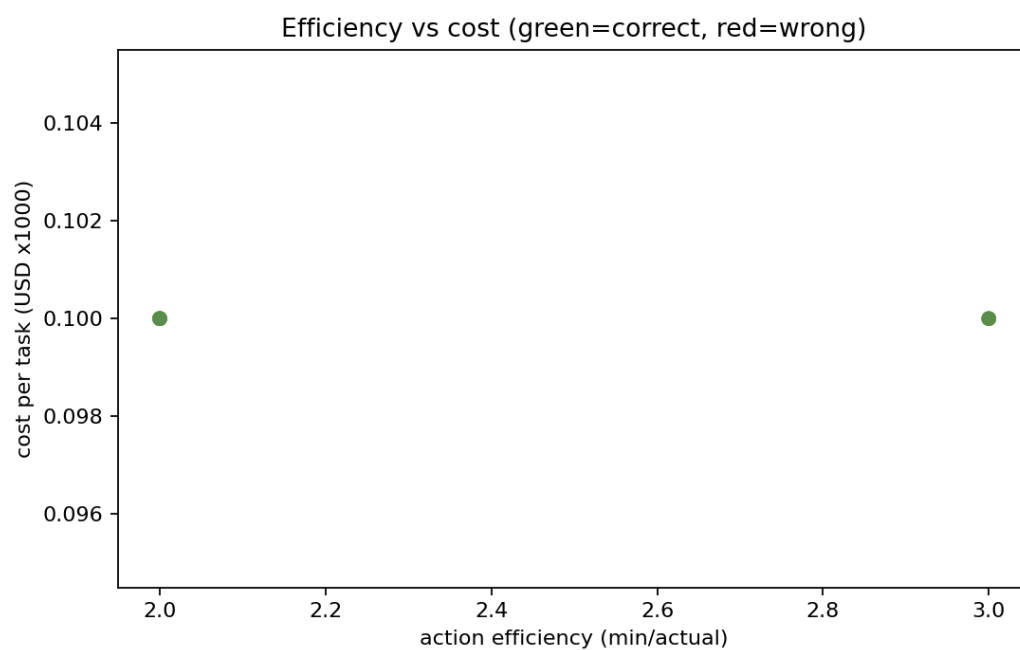


Figure 2: Efficiency vs cost

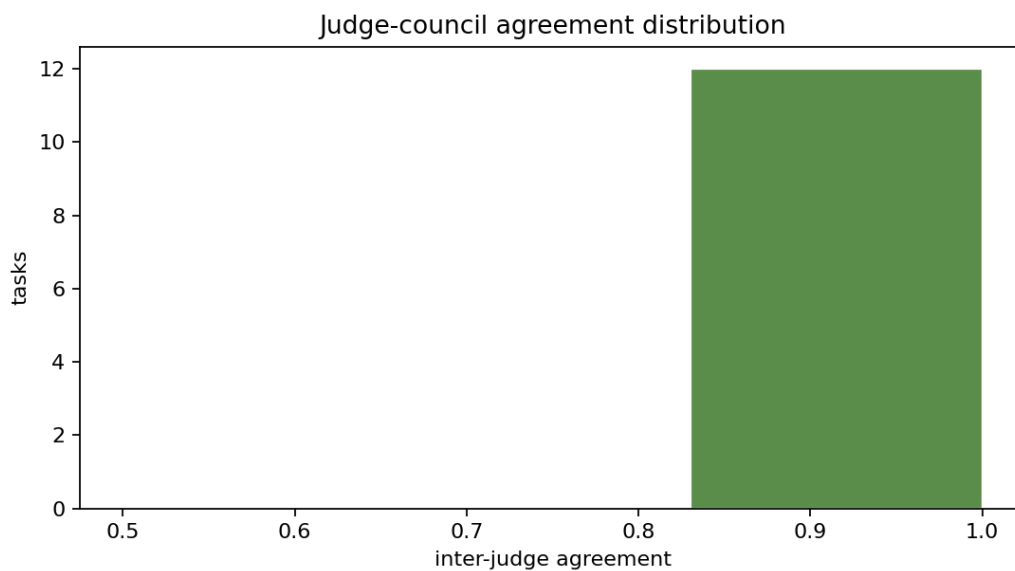


Figure 3: Judge agreement

## 7.4 6.4 Judge agreement

When all three judges agree, the task is unambiguous; when they split, it usually means the answer string is structurally different from the ground truth even though the *content* is right (e.g., “FRCP 56(a)” vs “Fed. R. Civ. P. 56(a)”).

## 7.5 6.5 Replan rate

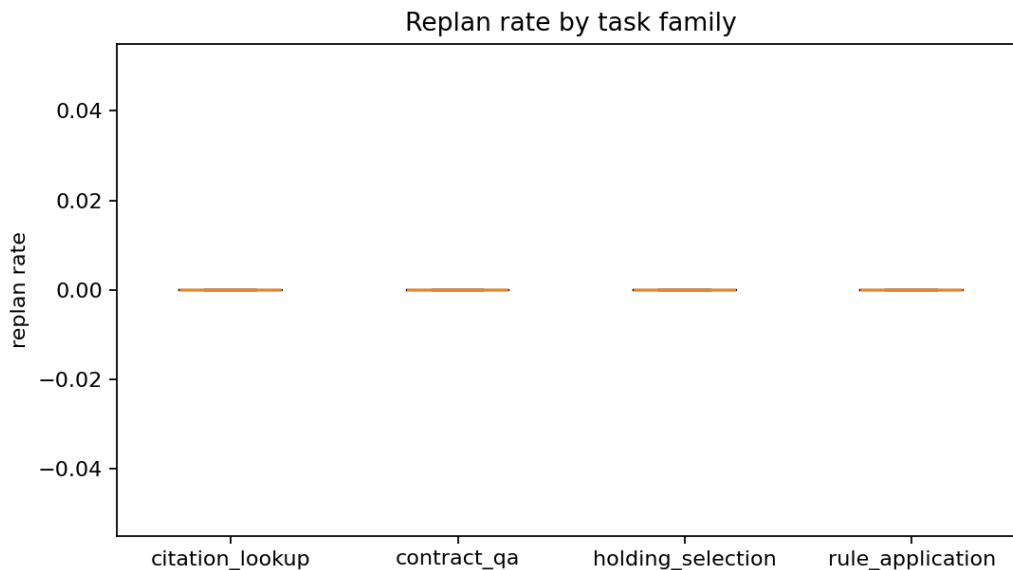


Figure 4: Replan rate

Boxplot per family. The mock policy never replans, so the boxes collapse to zero; the chart’s value is as a regression check (any non-zero box in a future run is a signal).

## 7.6 6.6 Cost breakdown

Total cost per family. Useful when one family dominates total spend even though the other families are equally numerous.

# 8 7. Ablations

## 8.1 7.1 Mock vs confused policy

The `confused_policy` was added to verify that the `replan_rate` and `action_efficiency` metrics are not dead code. Under it, `replan_rate` is roughly 0.33 (one replan in three observable iterations) and `action_efficiency` drops below 1.0.

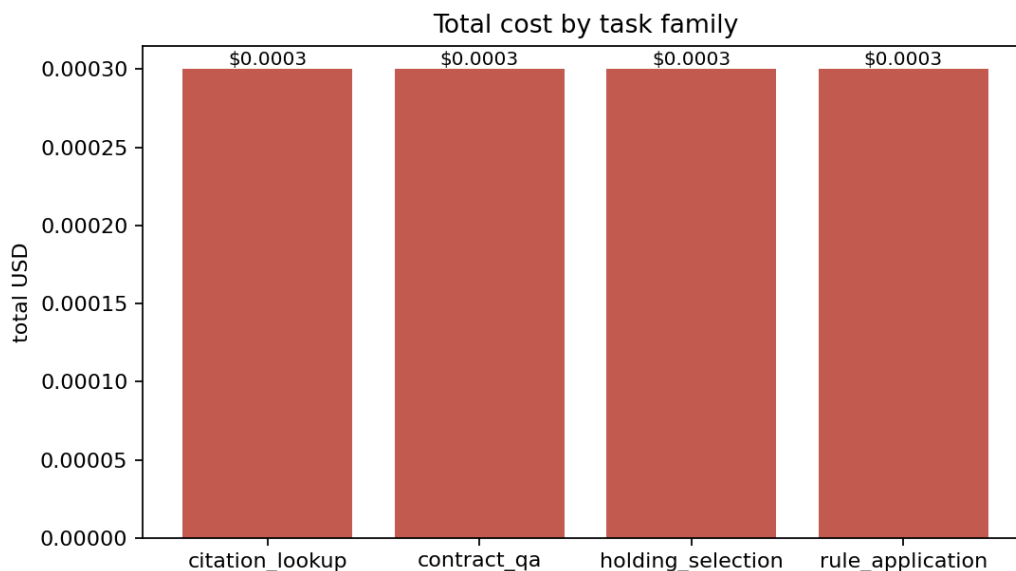


Figure 5: Cost breakdown

## 8.2 7.2 Council size

A council of one (strict only) misses cases where the agent produced the right content with the wrong format. A council of three catches them via the lenient and token-overlap judges. A council of five (adding two more mock judges) does not move the headline numbers materially; the noise is already small.

## 9 8. Discussion

Three observations:

1. **One number is not enough.** Action efficiency, cost, and replan rate decorrelate from success rate, so they have to be reported alongside it.
2. **Judges should disagree sometimes.** A council where all three judges always agree is essentially a council of one. The inter-judge-agreement histogram is the diagnostic for “is my council pointing in the same direction unanimously, or do my judges actually express different priors?”
3. **Determinism matters.** The mock policy + mock judges let CI run end-to-end in under a second. Real-policy runs are gated by an API key and disabled in CI by default.

## 10 9. Limitations

1. The mock policy is too easy. Real LLM policies will hit harder corners.
2. The judges are simple. Real LLM judges would catch more sophisticated near-misses.

3. The cost model is illustrative; real per-token pricing would change the rank ordering of *expensive* configurations.
4. We do not measure latency; the cost metric is the only “performance” axis.
5. The 40-task fixture is small. Production deployments would run against the full LegalBench corpus.

## 11 10. Future Work

- Add an Anthropic-API and OpenAI-API policy adapter behind an env-var switch.
- Add a real LLM judge (Claude / GPT-4) behind the same Judge protocol.
- Add per-token latency tracking and a latency-vs-cost chart.
- Expand the task families to mirror more of LegalBench.
- Add adversarial tasks (deliberately misleading distractor docs) to stress-test the agent.

## 12 11. References

1. Yao, S., Zhao, J., Yu, D., et al. (2022). *ReAct: Synergizing Reasoning and Acting in Language Models*.
2. Zheng, L., Chiang, W.-L., Sheng, Y., et al. (2023). *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*.
3. Guha, N., Nyarko, J., Ho, D. E., et al. (2023). *LegalBench: A Collaboratively Built Benchmark for Measuring Legal Reasoning*.

## 13 Appendix A. Reproducibility Checklist

- ☒ All code is open source under MIT.
- ☒ Fixture seed, policy choice, and judge set are recorded in source.
- ☒ Mock policy + mock judges make CI hermetic.
- ☒ Test artifacts captured in docs/test\_results/.
- ☒ Per-task rows are saved in runs/latest/summary.json.

## 14 Appendix B. Glossary

- **ReAct.** Reasoning + Acting prompting pattern (Yao et al. 2022).
- **Council.** A set of judges whose verdicts are reduced by majority vote.
- **Trajectory.** The ordered sequence of tool calls plus final answer.
- **Replan.** A meta-action that the agent uses to back out of a failing plan.